

HDOC – Experiment 21

Question

Menu-Driven Program Using Nested Loops and Switch-Case

❖ Understanding problem

Objective

The program accepts two positive integers m and n , where:

- $m > 0$
- $n > 0$
- $m \leq n$

It then repeatedly displays a menu until the user selects 5 (Exit).

Operations used

Choice	Operation
1	First 10 multiples of every integer from m to n
2	Addition table of every integer from m to n
3	Subtraction table of every integer from m to n
4	Division table of every integer from m to n , rounded to 2 decimal places
5	Exit
Other	Display invalid-choice error

❖ Algorithm

Step 1: Start.

Step 2: Declare integer variables m, n, choice, i, and j.

Step 3: Accept values of m and n.

Step 4: Check whether m and n are positive and $m \leq n$.

Step 5: If the values are invalid, display an error message and ask the user to enter m and n again. Repeat until valid values are entered.

Step 6: Display the following menu:

1. Multiplication tables
2. Addition tables
3. Subtraction tables
4. Division tables
5. Exit

Step 7: Accept the user's choice.

Step 8: Use a switch statement to perform the selected operation.

Step 9: If choice = 1:

- Use an outer loop from $i = m$ to n .
- For each i , use an inner loop from $j = 1$ to 10.
- Display $i \times j = i*j$.

Step 10: If choice = 2:

- Use an outer loop from $i = m$ to n .
- For each i , use an inner loop from $j = 1$ to 10.
- Display $i + j = i+j$.

Step 11: If choice = 3:

- Use an outer loop from $i = m$ to n .
- For each i , use an inner loop from $j = 1$ to 10.
- Display $i - j = i-j$.

Step 12: If choice = 4:

- Use an outer loop from $i = m$ to n .
- For each i , use an inner loop from $j = 1$ to 10.
- Divide i by j using floating-point division.
- Display the result rounded to two decimal places.

Step 13: If choice = 5, display an exit message and terminate the menu loop.

Step 14: For any other menu choice, display an appropriate error message.

Step 15: Repeat Steps 6–14 until the user selects option 5.

Step 16: Stop.

❖ Test case table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	$m=2, n=3, \text{choice}=1$	First 10 multiples of 2 and 3	First 10 multiples of 2 and 3 displayed	Pass
2	$m=2, n=3, \text{choice}=2$	Addition tables of 2 and 3 from addends 1–10	Correct addition tables displayed	Pass
3	$m=5, n=6, \text{choice}=3$	Subtraction tables of 5 and 6 using 1–10	Correct subtraction tables displayed	Pass
4	$m=5, n=6, \text{choice}=4$	Division tables of 5 and 6 using divisors 1–10, to 2 decimal places	Correct division results displayed	Pass
5	$m=4, n=4, \text{choice}=1$	First 10 multiples of 4	Multiples from $4 \times 1 = 4$ to $4 \times 10 = 40$	Pass
6	$m=1, n=1, \text{choice}=4$	Division results for 1, e.g. $1/3 = 0.33$	Results correctly displayed to 2 decimal places	Pass
7	$m=0, n=5$	Error because m is not positive; ask again	Error message displayed	Pass
8	$m=-2, n=5$	Error because m is negative; ask again	Error message displayed	Pass
9	$m=7, n=3$	Error because $m > n$; ask again	Error message displayed	Pass

10	m=2, n=5, choice=8	Invalid menu choice error	Error message displayed; menu shown again	Pass
11	m=2, n=5, choice=5	Exit program	Exit message displayed	Pass

❖ Program code in C

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int m, n, choice, i, j;
```

```
    /* Input and validation of m and n */
```

```
    do
```

```
    {
```

```
        printf("Enter two positive integers m and n (m <= n): ");
```

```
        scanf("%d %d", &m, &n);
```

```
        if (m <= 0 || n <= 0 || m > n)
```

```
        {
```

```
            printf("Error: Enter positive integers with m <= n.\n");
```

```
        }
```

```
    } while (m <= 0 || n <= 0 || m > n);
```

```
    /* Menu is repeated until the user selects Exit */
```

```

do
{
    printf("\n===== MENU =====\n");
    printf("1. Print first 10 multiples\n");
    printf("2. Print addition tables\n");
    printf("3. Print subtraction tables\n");
    printf("4. Print division tables\n");
    printf("5. Exit\n");
    printf("===== \n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            /* Multiplication tables */
            for (i = m; i <= n; i++)
            {
                printf("\nFirst 10 multiples of %d:\n", i);

                for (j = 1; j <= 10; j++)
                {
                    printf("%d x %d = %d\n", i, j, i * j);
                }
            }
            break;

```

case 2:

```
/* Addition tables */  
for (i = m; i <= n; i++)  
{  
    printf("\nAddition table of %d:\n", i);  
  
    for (j = 1; j <= 10; j++)  
    {  
        printf("%d + %d = %d\n", i, j, i + j);  
    }  
}  
break;
```

case 3:

```
/* Subtraction tables */  
for (i = m; i <= n; i++)  
{  
    printf("\nSubtraction table of %d:\n", i);  
  
    for (j = 1; j <= 10; j++)  
    {  
        printf("%d - %d = %d\n", i, j, i - j);  
    }  
}  
break;
```

case 4:

```
/* Division tables */
```

```
    for (i = m; i <= n; i++)
    {
        printf("\nDivision table of %d:\n", i);

        for (j = 1; j <= 10; j++)
        {
            printf("%d / %d = %.2f\n",
                i, j, (float)i / j);
        }
    }
    break;

case 5:
    printf("\nExiting the program...\n");
    break;

default:
    printf("\nError: Invalid menu choice. "
        "Please enter a choice from 1 to 5.\n");
}

} while (choice != 5);

return 0;
}
```

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp21]
$ vi accept.c

(kali㉿kali)-[~/Desktop/exp21]
$ gcc accept.c -o accept

(kali㉿kali)-[~/Desktop/exp21]
$ ./accept
Enter two positive integers m and n ( $m \leq n$ ): 2 3

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====
Enter your choice: 1

First 10 multiples of 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
```

```
First 10 multiples of 3:
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====
Enter your choice: 2

Addition table of 2:
2 + 1 = 3
2 + 2 = 4
2 + 3 = 5
2 + 4 = 6
2 + 5 = 7
2 + 6 = 8
2 + 7 = 9
2 + 8 = 10
2 + 9 = 11
2 + 10 = 12
```



```

Subtraction table of 5:
5 - 1 = 4
5 - 2 = 3
5 - 3 = 2
5 - 4 = 1
5 - 5 = 0
5 - 6 = -1
5 - 7 = -2
5 - 8 = -3
5 - 9 = -4
5 - 10 = -5

Subtraction table of 6:
6 - 1 = 5
6 - 2 = 4
6 - 3 = 3
6 - 4 = 2
6 - 5 = 1
6 - 6 = 0
6 - 7 = -1
6 - 8 = -2
6 - 9 = -3
6 - 10 = -4

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====
Enter your choice: 4

```

```

Division table of 5:
5 / 1 = 5.00
5 / 2 = 2.50
5 / 3 = 1.67
5 / 4 = 1.25
5 / 5 = 1.00
5 / 6 = 0.83
5 / 7 = 0.71
5 / 8 = 0.62
5 / 9 = 0.56
5 / 10 = 0.50

Division table of 6:
6 / 1 = 6.00
6 / 2 = 3.00
6 / 3 = 2.00
6 / 4 = 1.50
6 / 5 = 1.20
6 / 6 = 1.00
6 / 7 = 0.86
6 / 8 = 0.75
6 / 9 = 0.67
6 / 10 = 0.60

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====
Enter your choice: 5

Exiting the program...

```

```

Addition table of 3:
3 + 1 = 4
3 + 2 = 5
3 + 3 = 6
3 + 4 = 7
3 + 5 = 8
3 + 6 = 9
3 + 7 = 10
3 + 8 = 11
3 + 9 = 12
3 + 10 = 13

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====

Enter your choice: 5

Exiting the program...

(kali@kali)-[~/Desktop/exp21]
$ ./accept
Enter two positive integers m and n ( $m \leq n$ ): 5 6

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====

Enter your choice: 3

```

```

(kali@kali)-[~/Desktop/exp21]
$ ./accept
Enter two positive integers m and n ( $m \leq n$ ): 4 4

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====

Enter your choice: 1

First 10 multiples of 4:
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====

Enter your choice: 5

Exiting the program...

```

```

(kali@kali)-[~/Desktop/exp21]
$ ./accept
Enter two positive integers m and n ( $m \leq n$ ): 1 1

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====
Enter your choice: 4

Division table of 1:
1 / 1 = 1.00
1 / 2 = 0.50
1 / 3 = 0.33
1 / 4 = 0.25
1 / 5 = 0.20
1 / 6 = 0.17
1 / 7 = 0.14
1 / 8 = 0.12
1 / 9 = 0.11
1 / 10 = 0.10

===== MENU =====
1. Print first 10 multiples
2. Print addition tables
3. Print subtraction tables
4. Print division tables
5. Exit
=====
Enter your choice: 5

Exiting the program...

```

```

(kali@kali)-[~/Desktop/exp21]
$ ./accept
Enter two positive integers m and n ( $m \leq n$ ): 0 5
Error: Enter positive integers with  $m \leq n$ .
Enter two positive integers m and n ( $m \leq n$ ): -2 5
Error: Enter positive integers with  $m \leq n$ .
Enter two positive integers m and n ( $m \leq n$ ): 7 3
Error: Enter positive integers with  $m \leq n$ .
Enter two positive integers m and n ( $m \leq n$ ): 

```